Challenge final: Testing con Cypress

De qué se trata

En este trabajo final van a poner en práctica todo lo que aprendieron en el curso sobre una aplicación web de práctica que tiene varios módulos, flujos más completos y **errores reales para encontrar**. Trabajan en grupo, como lo haría un equipo de QA en un proyecto real: primero planifican, después automatizan y por último reportan los bugs que encuentren.

Qué tienen que entregar

1. **Plan de casos de prueba** en Excel o Google Sheets.
2. **Proyecto de Cypress** con los tests automatizados.
3. **Tablero de Trello** con los bugs encontrados.
4. **Repositorio de GitHub público** con un README que enlace todo lo anterior.

El sitio a testear: Toolshop

Toolshop es una tienda online de herramientas hecha para practicar testing. Tiene catálogo de productos, carrito, registro, login, checkout, formulario de contacto y una API. Es la misma aplicación que usamos en la clase de APIs.

- **Link del sitio:** https://with-bugs.practicesoftwaretesting.com/ · API: https://api-with-bugs.practicesoftwaretesting.com/api/documentation

Usuario de prueba: `customer@practicesoftwaretesting.com` con contraseña `welcome01` (también existe `customer2@practicesoftwaretesting.com`).

Antes de empezar, tengan en cuenta

- **La base de datos es compartida** con todas las personas que usan el sitio. No carguen datos personales: inventen nombres, mails y teléfonos.
- **Los datos pueden cambiar o reiniciarse.** Sus tests no pueden depender de algo que creaste ayer ni de lo que haya hecho otro grupo.
- Si un test registra un usuario, el mail tiene que ser **único en cada ejecución** (por ejemplo, generado con la fecha: `user${Date.now()}@test.com`).

Cómo conviene organizarse

1. Exploren el sitio a mano, como si fueran usuarios, y anoten lo que les parezca raro.
2. Armen el plan de casos de prueba.
3. Automaticen los flujos pedidos.
4. Reporten los bugs en Trello a medida que los encuentren.
5. Completen el README y revisen todo antes de entregar.

Parte 1: Plan de casos de prueba

Antes de escribir código, documenten **qué van a probar**. Usen el template de casos de prueba que vimos en clase: [link al template].

Mínimo: 12 bugs por grupo. El sitio tiene muchos errores, así que van a tener material de sobra.

Clasificación de los casos

Cada caso tiene que ser de uno de estos tres tipos, y el plan tiene que tener de los tres (te sugerimos al menos 3 de cada uno):

- **Positivo:** todo se hace bien y el sitio responde como corresponde. Ejemplo: iniciar sesión con el usuario de prueba.
- **Negativo:** se ingresan datos incorrectos o se deja algo sin completar, y el sitio tiene que avisar. Ejemplo: iniciar sesión con una contraseña equivocada.
- **De borde:** se prueban los límites. Ejemplo: un mensaje de contacto muy largo, un campo con un solo carácter o una cantidad 0 en el carrito.

Parte 2: Proyecto de Cypress

Instalación

1. Creen una carpeta para el proyecto y ábranla en la terminal.
2. Inicien el proyecto con `npm init -y`.
3. Instalen Cypress con `npm install cypress --save-dev`.
4. Abran Cypress con `npx cypress open`.

Estructura mínima

Los tests se separan en **varios archivos, uno por cada flujo de la Parte 3**, todos dentro de `cypress/e2e/`. Ejemplo de estructura:

```
None
cypress/
  e2e/
    01-compra.cy.js        // 3.1 Compra exitosa
    02-registro.cy.js      // 3.2 Validaciones del registro
    03-contacto.cy.js      // 3.3 Formulario de contacto
    04-login.cy.js         // 3.4 Inicio de sesión con datos
incorrectos
    05-busqueda.cy.js      // 3.5 Búsqueda de productos
    06-carrito.cy.js       // 3.6 Eliminar un producto del carrito
    07-api.cy.js           // 3.7 Prueba de API
  fixtures/
    users.json             // datos de prueba
    otro.json
```

```
  support/
    commands.js            // custom commands
    otrosComandos.js
    e2e.js                 // importa commands.js (ya viene por
defecto)
```

Lo que sí o sí tiene que tener el proyecto

- **Al menos 2 custom commands** en `commands.js` (por ejemplo, uno para iniciar sesión y otro para completar un formulario).
- **Al menos 1 fixture** con los datos de prueba (usuarios, datos del formulario de contacto, etc.). Los datos no se escriben a mano dentro de los tests.
- **Selectores claros.** Si el elemento tiene un atributo `data-test`, usenlo. Pueden verlo inspeccionando la página.
- Un archivo `.gitignore` que incluya `node_modules`, para no subir esa carpeta al repositorio.

Parte 3: Flujos a automatizar

3.1 Compra exitosa

1. Entrar a la página principal y verificar que se muestran productos.
2. Abrir un producto y agregarlo al carrito.
3. Ir al carrito y verificar que el producto está ahí.
4. Iniciar sesión con el usuario de prueba (sus datos salen del fixture).
5. Avanzar por el checkout hasta confirmar la compra. Si el sitio pide datos de pago, usen datos de prueba.
6. Verificar que aparece el mensaje de confirmación.

3.2 Validaciones del formulario de registro

1. Ir a la página de registro.
2. Intentar enviar el formulario sin completar ningún campo.
3. Verificar que aparecen los mensajes de error correspondientes.
4. Verificar que **no se creó ninguna cuenta** (por ejemplo, que seguimos en la página de registro).

3.3 Formulario de contacto

1. Ir a la página de contacto.
2. Completar todos los campos que pide el formulario con datos válidos (salen del fixture).
3. Enviar el mensaje.
4. Verificar que se muestra la confirmación.

3.4 Inicio de sesión con datos incorrectos

1. Ir a la página de inicio de sesión.
2. Ingresar el email del usuario de prueba con una contraseña equivocada (los datos salen del fixture).
3. Enviar el formulario.
4. Verificar que aparece un mensaje de error.
5. Verificar que **no se inició sesión** (por ejemplo, que seguimos en la página de inicio de sesión).

3.5 Búsqueda de productos

1. Entrar a la página principal.
2. Escribir en el buscador el nombre de una herramienta que veas en el catálogo y buscar.
3. Verificar que aparecen resultados.
4. Verificar que el nombre de un producto del resultado contiene lo que buscaste.

3.6 Eliminar un producto del carrito

1. Entrar a la página principal, abrir un producto y agregarlo al carrito.
2. Ir al carrito y verificar que el producto está.
3. Eliminar el producto.
4. Verificar que el carrito queda vacío.

3.7 Prueba de API

Este test se hace con `cy.request`, sin abrir la página. Los endpoints están en la documentación de la API (Swagger) que figura al principio. Los dos pedidos van dentro del mismo `it`.

1. Pedir la lista de productos y verificar que el status es 200 y que la lista no está vacía.
2. Iniciar sesión por API con el usuario de prueba y verificar que el status es 200 y que la respuesta trae un token de acceso.

Requisitos generales

- **Los tests tienen que ser independientes.** Cada `it` debe poder ejecutarse solo, sin depender de que otro haya corrido antes.
- **Sin repetición innecesaria.** Si escribís las mismas líneas en varios tests (por ejemplo, iniciar sesión o agregar un producto al carrito), convertilas en un custom command.
- Usen los comandos y métodos que vimos durante el curso. Pueden sumar cualquier herramienta o técnica extra que les sirva.
- Si quieren ir más allá de lo mínimo, exploren el panel de administración del sitio y agreguen casos de prueba adicionales.

Parte 4: Reporte de bugs en Trello

Cada error que encuentren se reporta en un tablero de Trello: [link al tablero]. Usen el template de reporte de bugs que vimos en clase: [link al template].

Columnas del tablero

Backlog de bugs → En progreso → Listo para revisar → Cerrado.

Una regla para que el reporte sirva

Otra persona tiene que poder repetir el error **leyendo solo la tarjeta**. Si hace falta adivinar un paso o un dato, el reporte está incompleto.

Cada caso de prueba que dio *Fail* en el plan tiene que enlazar a su tarjeta de Trello, y cada tarjeta tiene que poder rastrearse hasta su caso de prueba.

Cantidad mínima de bugs a reportar: 12 por equipo.

Parte 5: Entrega y evaluación

Cómo se entrega

- **Qué:** el link al repositorio de GitHub.
- **Cuándo:** 13/11/2026.
- **Estado del repositorio:** tiene que ser **público**.
- **Quién:** entrega **un solo integrante por grupo**.
- **Certificado:** tienen que completar **sí o sí** el formulario para la obtención del certificado.

Qué tiene que tener el repositorio

1. El proyecto de Cypress, con los archivos de tests, los fixtures y los custom commands.
2. Un archivo `README.md` con:
    - el link al tablero de Trello,
    - el link al archivo de casos de prueba,
    - los nombres de las personas del grupo y el mail de cada una.

Checklist antes de entregar

- El repositorio es público (probá abrirlo en una ventana de incógnito).
- El link al Excel o Google Sheets abre sin pedir permisos.
- El link a Trello abre sin pedir permisos.
- Clonaste el repositorio en otra carpeta, corriste `npm install` y los tests funcionan.
- Hay un archivo de tests por cada flujo de la Parte 3, al menos 2 custom commands y 1 fixture.
- Cada bug de Trello tiene todos los campos y su evidencia.
- El README tiene los nombres y mails de todo el grupo.

Qué vamos a evaluar

Área	Qué miramos
Comprensión y aplicación	Que usen bien los conceptos del curso. Carpetas y archivos bien estructurados. Código limpio y ordenado.
Casos de prueba	Un plan claro y detallado, con buena cobertura de casos positivos, negativos y de borde.
Automatización	Que cumplan los siete flujos de la Parte 3, con aserciones adecuadas en cada test, tests independientes y sin código repetido.
Reporte de bugs	Que usen el template correctamente y que cada defecto esté descripto con claridad y detalle.